DEBEZIUM CDC POC

Debezium Kafka Connect and Oracle Source Connector Setup

Kafka Connect infrastructure and Oracle source connector guide

1. Overview

This document covers the Kafka Connect infrastructure and Debezium Oracle Source Connector implementation for the Debezium CDC POC.

It assumes that Oracle Database preparation has already been completed according to:

`Oracle_CDC_Source_Preparation_for_Debezium.txt`

The objective is to configure Kafka Connect, register the Oracle source connector, capture the initial snapshot, and verify real-time database changes through Kafka.

Architecture

```
Oracle Database (CDCDB)
        |
        | LogMiner
        v
Debezium Oracle Source Connector
        |
        v
Kafka Connect
        |
        v
Apache Kafka
        |
        v
oracle-cdc.CDC_ADMIN.CUSTOMERS
```

2. Environment

Component	Configuration
Docker Engine	29.6.1
Docker Compose	5.1.4
Apache Kafka	4.1.2
Kafka Connect / Debezium	3.5.2.Final image
Kafka Connect REST API	8083
Kafka external listener	9092
Kafka internal listener	29092
Oracle hostname	oracle-poc
Oracle port	1521

Component	Configuration
Oracle PDB	CDCDB

3. Docker Compose Configuration

The Kafka infrastructure is defined in:

`docker/kafka/compose.yaml`

It contains two services:

- `kafka`: Apache Kafka broker running in KRaft mode.
- `connect`: Debezium Kafka Connect runtime.

The containers are:

Container	Purpose
`cdc-kafka`	Kafka broker
`debezium-connect`	Kafka Connect runtime

Oracle runs in a separate container:

`oracle-poc`

3.1 Start Kafka infrastructure

From the project root:

```
docker compose -f docker/kafka/compose.yaml up -d
```

Check running containers:

```
docker ps
```

3.2 Verify Kafka broker

```
docker run --rm --network kafka_default apache/kafka:4.1.2
    /opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server kafka:29092
```

This verifies that the broker is reachable over the internal Docker network.

3.3 Verify Kafka Connect REST API

```
Invoke-RestMethod -Uri "http://localhost:8083/" -Method Get
```

The REST API provides information about the Kafka Connect runtime and its Kafka cluster connection.

4. Docker Network Configuration

The Oracle container must be reachable from Kafka Connect.

The Kafka Compose network is:

`kafka_default`

Connect Oracle to this network:

```
docker network connect kafka_default oracle-poc
```

Verify network membership:

```
docker network inspect kafka_default
```

The following containers should be present:

- `oracle-poc`
- `cdc-kafka`
- `debezium-connect`

4.1 Verify Oracle connectivity from Kafka Connect

```
docker exec debezium-connect bash -c 'timeout 5 bash -c "</dev/tcp/oracle-poc/1521" && echo "Oracle
    reachable" || echo "Connection failed"'
```

Expected result:

```
Oracle reachable
```

This confirms that Kafka Connect can reach the Oracle listener using the Docker hostname.

5. Verify Debezium Connector Plugins

Kafka Connect supports multiple connector plugins.

List the installed plugins:

```
Invoke-RestMethod -Uri "http://localhost:8083/connector-plugins" -Method Get
```

The following connectors were verified:

```
io.debezium.connector.oracle.OracleConnector
io.debezium.connector.jdbc.JdbcSinkConnector
```

The Oracle connector version was:

```
3.5.2.Final
```

The JDBC Sink Connector is also available for the next implementation phase.

5.1 Inspect Oracle connector configuration properties

```
Invoke-RestMethod -Uri
    "http://localhost:8083/connector-plugins/io.debezium.connector.oracle.OracleConnector/config"
    -Method Get
```

This endpoint exposes the connector's supported configuration properties.

6. Oracle JDBC Driver Verification

The Oracle connector plugin directory already contains the JDBC driver and its supporting libraries.

The included driver:

```
ojdbc11-21.15.0.0.jar
```

Inspect the plugin directory:

```
docker run --rm quay.io/debezium/connect:3.5.2.Final ls -lh /kafka/connect/debezium-connector-oracle
```

No additional JDBC driver installation was required for the configured connector.

7. Oracle Source Connector Configuration

7.1 Configuration template

The configuration template is stored in:

`debezium/oracle-source.template.json`

```
{
  "name": "oracle-source-connector",
  "config": {
    "connector.class": "io.debezium.connector.oracle.OracleConnector",
    "tasks.max": "1",

    "database.hostname": "oracle-poc",
    "database.port": "1521",
    "database.user": "C##DBZUSER",
    "database.password": "__INJECT_AT_RUNTIME__",

    "database.dbname": "FREE",
    "database.pdb.name": "CDCDB",

    "database.connection.adapter": "logminer",
    "topic.prefix": "oracle-cdc",
    "snapshot.mode": "initial",

    "schema.include.list": "CDC_ADMIN",
    "table.include.list": "CDC_ADMIN.CUSTOMERS",

    "schema.history.internal.kafka.bootstrap.servers": "kafka:29092",
    "schema.history.internal.kafka.topic": "schema-changes.oracle-cdc"
  }
}
```

7.2 Configuration explanation

Property	Purpose
`connector.class`	Selects the Debezium Oracle connector
`tasks.max`	Maximum number of connector tasks
`database.hostname`	Oracle Docker hostname
`database.port`	Oracle listener port
`database.user`	CDC capture account
`database.password`	Injected at runtime
`database.dbname`	Oracle CDB name
`database.pdb.name`	Target PDB
`database.connection.adapter`	Uses LogMiner
`topic.prefix`	Kafka topic namespace
`snapshot.mode`	Initial snapshot strategy
`schema.include.list`	Included Oracle schema
`table.include.list`	Included source table
`schema.history.internal.kafka.bootstrap.servers`	Kafka broker for schema history
`schema.history.internal.kafka.topic`	Schema history topic

7.3 Sensitive configuration handling

The actual connector configuration containing the Oracle password must never be committed to Git.

The template uses:

```
__INJECT_AT_RUNTIME__
```

The local file:

`debezium/oracle-source.json`

is excluded through `.gitignore`.

The real password is injected into the PowerShell configuration object before sending the connector configuration to Kafka Connect.

8. Secure Runtime Password Injection

Load the JSON template:

```
$config = Get-Content debezium/oracle-source.template.json -Raw | ConvertFrom-Json
```

Add the connector name to the configuration object:

```
$config.config | Add-Member -MemberType NoteProperty -Name "name" -Value $config.name -Force
```

Read the password securely:

```
$securePassword = Read-Host "Enter C##DBZUSER password" -AsSecureString

$pointer = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($securePassword)

$plainPassword = [Runtime.InteropServices.Marshal]::PtrToStringBSTR($pointer)
```

Inject the password into the in-memory configuration:

```
$config.config.'database.password' = $plainPassword
```

The password is not written into the template file.

> **Note:** The connector configuration is stored by Kafka Connect. Runtime injection prevents committing the password to Git, but it does not mean the password is never stored by the Connect infrastructure.

9. Connector Configuration Validation

Before registration, validate the connector configuration.

Convert the configuration object to JSON:

```
$body = $config.config | ConvertTo-Json -Depth 10
```

Send the validation request:

```
$validation = Invoke-RestMethod `
    -Uri
        "http://localhost:8083/connector-plugins/io.debezium.connector.oracle.OracleConnector/config/validate"
        `
    -Method Put `
    -ContentType "application/json" `
    -Body $body
```

Check validation errors:

```
"Total validation errors: $($validation.error_count)"
```

Verified result:

```
Total validation errors: 0
```

The Oracle connector configuration passed validation.

10. Register Oracle Source Connector

Create the registration request using the complete configuration wrapper:

```
$body = $config | ConvertTo-Json -Depth 10
```

Register the connector:

```
Invoke-RestMethod `
    -Uri "http://localhost:8083/connectors" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body
```

Expected connector name:

```
oracle-source-connector
```

The connector was successfully registered.

11. Verify Connector Status

Check the connector status:

```
Invoke-RestMethod `
    -Uri "http://localhost:8083/connectors/oracle-source-connector/status" `
    -Method Get |
    ConvertTo-Json -Depth 10
```

Verified status:

```
connector.state = RUNNING
tasks[0].state = RUNNING
```

Both the connector and its task reached the RUNNING state.

12. Verify Kafka Topics

List all Kafka topics:

```
docker exec cdc-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:29092 --list
```

The following topics were observed:

```
__consumer_offsets
connect-configs
connect-offsets
connect-status
oracle-cdc
oracle-cdc.CDC_ADMIN.CUSTOMERS
schema-changes.oracle-cdc
```

The source table data is published to:

```
oracle-cdc.CDC_ADMIN.CUSTOMERS
```

The schema history topic is:

```
schema-changes.oracle-cdc
```

13. Initial Snapshot Verification

The connector was configured with:

```
"snapshot.mode": "initial"
```

This causes Debezium to capture the existing records when the connector performs its initial snapshot.

Consume the first three records:

```
docker exec cdc-kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka:29092
    --topic oracle-cdc.CDC_ADMIN.CUSTOMERS --from-beginning --max-messages 3
```

Verified result:

```
Processed a total of 3 messages
```

The three existing CUSTOMERS records were successfully captured.

The snapshot event operation is:

```
{
  "op": "r"
}
```

Where:

- `r` = Read / snapshot record.

14. Real-Time CDC UPDATE Verification

After confirming the initial snapshot, a real-time update was performed against the Oracle source table.

14.1 Execute an UPDATE in Oracle

Connect using the table owner account:

```
docker exec -it oracle-poc sqlplus CDC_ADMIN@//localhost:1521/CDCDB
```

Execute:

```
UPDATE CDC_ADMIN.CUSTOMERS
SET PHONE = '01011112222',
    UPDATED_AT = SYSTIMESTAMP
WHERE CUSTOMER_ID = 1;

COMMIT;
```

Verified Oracle output:

```
1 row updated.
Commit complete.
```

Exit SQL*Plus:

```
EXIT;
```

14.2 Consume snapshot and update events

Use a new consumer group to read the topic from the beginning:

```
docker exec cdc-kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka:29092
    --topic oracle-cdc.CDC_ADMIN.CUSTOMERS --group cdc-update-verification --from-beginning
    --max-messages 4
```

Expected messages:

- Three initial snapshot records.
- One real-time UPDATE record.

14.3 Verified UPDATE event

The actual event contained the following important information:

```
{
  "payload": {
    "before": {
      "CUSTOMER_ID": 1,
      "PHONE": "01012345678"
    },
    "after": {
      "CUSTOMER_ID": 1,
      "PHONE": "01011112222"
    },
    "source": {
      "connector": "oracle",
```

```
      "db": "CDCDB",
      "schema": "CDC_ADMIN",
      "table": "CUSTOMERS",
      "user_name": "CDC_ADMIN",
      "snapshot": "false"
    },
    "op": "u"
  }
}
```

The actual event also included schema metadata, timestamps, transaction information, and Oracle SCN details.

14.4 Event operation codes

Operation	Meaning
`r`	Snapshot read
`c`	Create / insert
`u`	Update
`d`	Delete

14.5 Result

The UPDATE event was successfully detected and published to Kafka.

The event confirmed:

- The old value was available in `before`.
- The new value was available in `after`.
- The operation code was `u`.
- The event was generated from a live change, not an initial snapshot.
- The source metadata identified the correct Oracle database, schema, table, and user.

> **Milestone completed: Oracle CDC source connector successfully captures initial snapshots and real-time UPDATE events.**

15. Troubleshooting

15.1 HTTP 405 during connector validation

The connector validation endpoint requires HTTP PUT.

Use:

```
-Method Put
```

15.2 Connector validation: missing name

The validation request must include the connector name.

Add it to the configuration object:

```
$config.config | Add-Member -MemberType NoteProperty -Name "name" -Value $config.name -Force
```

15.3 Oracle connection failure

Check:

1. Oracle container is running.
2. Oracle PDB service is available.
3. Kafka Connect and Oracle share the same Docker network.
4. The connector hostname is `oracle-poc`.
5. The CDC account can authenticate.

15.4 ORA-41900 during UPDATE

Error:

```
ORA-41900: missing UPDATE privilege on "CDC_ADMIN"."CUSTOMERS"
```

Cause:

The UPDATE was attempted using `C##DBZUSER`, the Debezium capture account.

Resolution:

Use the table owner account, `CDC_ADMIN`, for test data modifications.

15.5 Missing Oracle JDBC driver

The Oracle connector plugin directory already contained:

```
ojdbc11-21.15.0.0.jar
```

An additional downloaded JDBC driver was not required.

16. Current Status

Task	Status
Kafka Docker infrastructure	**Completed**
Kafka Connect runtime	**Completed**
Docker networking	**Verified**
Oracle connectivity from Connect	**Verified**
Oracle connector plugin	**Verified**
Oracle JDBC driver	**Verified**
Oracle connector configuration	**Validated**
Connector registration	**Completed**
Connector status	**RUNNING**
Kafka topics	**Verified**
Initial snapshot	**Verified**
Real-time UPDATE event	**Verified**
INSERT event test	**Pending**
DELETE event test	**Pending**

Task	Status
MySQL JDBC Sink Connector	**Pending**

17. Next Implementation Phase

The next phase is to implement Kafka-to-MySQL replication using the Debezium JDBC Sink Connector.

Planned work:

1. Prepare the MySQL target database and table.
2. Configure the JDBC Sink Connector.
3. Configure primary-key handling.
4. Implement insert and upsert behavior.
5. Verify update propagation.
6. Verify delete propagation.
7. Validate end-to-end Oracle-to-MySQL replication.

Final Architecture Goal

```
Oracle Database
      |
      v
Debezium Oracle Source Connector
      |
      v
Apache Kafka
      |
      v
Debezium JDBC Sink Connector
      |
      v
MySQL Database
```

Project: Debezium CDC POC
Repository: https://github.com/MoustafaBadr/debezium-cdc-poc
Document scope: Kafka Connect infrastructure and Oracle Source Connector
Current milestone: Initial snapshot and real-time UPDATE capture verified.